Estrutura do banco no Supabase

Guia de estudo e treino para a apresentação do ParkSim

Centro Universitário UNA – Barreiro · Banco de Dados · Desafio A3
Trilha Oracle Academy: Database Design · Professores Alexandre e Silvia
Grupo 5 · Novembro / 2026

Projeto Supabase: "Lebot's Project" · PostgreSQL 17 · região us-east-2
Site: lebot007.github.io/a3controlecarros · Repositório: github.com/Lebot007/a3controlecarros

COMECE POR AQUI

O banco em uma página

Este guia explica **tudo o que existe no Supabase do ParkSim** para que qualquer integrante consiga mostrar e defender o banco na apresentação. Todos os números e resultados abaixo foram obtidos executando o arquivo `database/parksim_schema.sql` do repositório em um PostgreSQL, ou seja, refletem exatamente o que o script cria.

6 **tabelas** tipo_vaga, cliente, funcionario, vaga, veiculo, movimentacao	**33** **colunas** somando as 6 tabelas	**6** **chaves primárias** uma por tabela (serial / bigserial)
7 **chaves estrangeiras** 5 entre tabelas do projeto + 2 para auth.users	**6** **UNIQUE** descricao, cpf (x2), email, numero, placa	**3** **CHECK** valor_hora ≥ 0, status, saída ≥ entrada
17 **índices** 6 de PK + 6 de UNIQUE + 5 criados à mão	**1** **function** fn_calcular_saida() em PL/pgSQL	**1** **trigger** trg_calcular_saida na tabela movimentacao
10 **políticas RLS** RLS ligado nas 6 tabelas	**0** **views / edge functions** não usados no projeto	**1** **usuário de teste** admin@parksim.com (Supabase Auth)

A ideia do projeto em 3 frases

- **Organização:** ParkSim, estacionamento fictício com 8 vagas de 4 tipos (Comum R$ 6/h, Coberta R$ 9/h, PCD e Idoso gratuitas).
- **Problema:** controle em papel/planilha gera duplicidade, erro no cálculo da cobrança e nenhum relatório confiável.
- **Solução:** banco relacional (PostgreSQL no Supabase) normalizado até a 3FN, com cobrança calculada pelo próprio banco (trigger), dados separados por conta (RLS) e um site que simula entradas e saídas em tempo real.

Como estudar: leia as seções 2 a 6 para entender a estrutura; use a seção 9 para treinar as perguntas que o professor pode fazer; a seção 10 sugere como dividir a fala entre os integrantes.

SEÇÃO 2

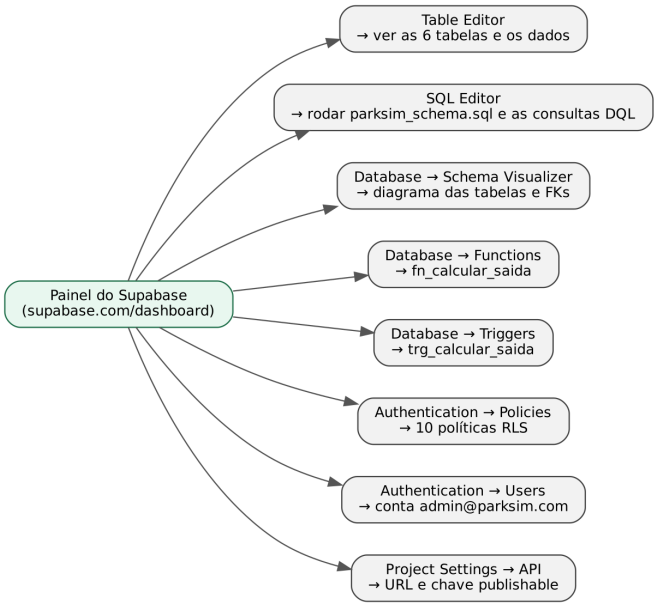
O que é o Supabase e o que usamos dele

O Supabase é uma plataforma em nuvem que entrega um **PostgreSQL completo** e, em volta dele, serviços prontos: login de usuários, uma API REST gerada automaticamente a partir das tabelas e um painel web para administrar tudo. Para a disciplina, o que importa é: **o banco é PostgreSQL de verdade** — os comandos DDL, DML e DQL do script são SQL padrão.

Recurso do Supabase	Usamos?	Para quê no ParkSim
Database (PostgreSQL 17)	Sim	As 6 tabelas, chaves, function, trigger e índices.
SQL Editor	Sim	Onde o script parksim_schema.sql foi executado e onde rodamos as consultas DQL.
Authentication (Auth)	Sim	Login por e-mail/senha. Cria a tabela interna auth.users e a função auth.uid().
API REST (PostgREST)	Sim	O site (JavaScript) lê e grava nas tabelas por essa API; cada chamada vira SQL no banco.
Row Level Security (RLS)	Sim	10 políticas que decidem quem pode ler/gravar cada linha.
Edge Functions	Não	Nenhuma (confirmado no projeto: lista vazia). Toda regra fica no banco.
Storage / Realtime	Não	O site não usa arquivos nem canais realtime; o painel é atualizado pelo próprio JavaScript.

Onde encontrar cada coisa no painel do Supabase

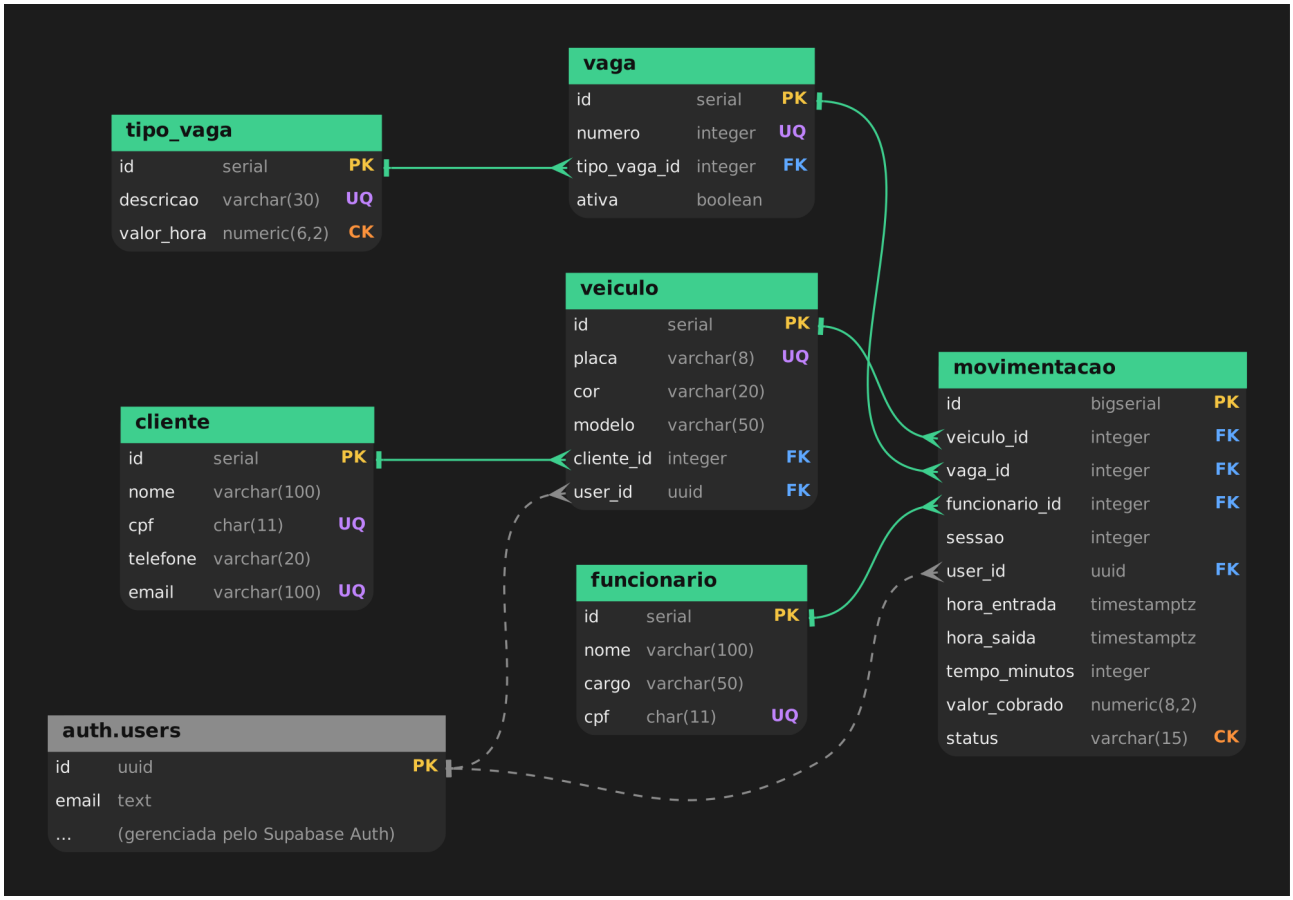
Durante a apresentação, abram o projeto em supabase.com/dashboard e sigam este mapa:



SEÇÃO 3

A estrutura completa (visão Schema Visualizer)

Diagrama gerado a partir do banco criado pelo script, no mesmo estilo do *Schema Visualizer* do Supabase (Database → Schema Visualizer). Cada caixa é uma tabela; as linhas verdes são chaves estrangeiras (o "pé de galinha" fica do lado **N**). As linhas tracejadas ligam à tabela `auth.users`, que é do Supabase Auth, não do nosso script.



Sigla	Significado
PK	Primary key — identifica cada linha de forma única
FK	Foreign key — aponta para a PK de outra tabela (integridade referencial)
UQ	UNIQUE — não deixa repetir o valor (ex.: dois clientes com o mesmo CPF)
CK	CHECK — regra de validação do valor

SEÇÃO 4

As 6 tabelas, uma a uma

tipo_vaga

Tabela de referência com o tipo de vaga e a **tarifa por hora**. Existe para a tarifa ficar em um só lugar (3FN): mudar o preço da vaga Comum altera 1 linha.

Coluna	Tipo	Restrições	Significado
id	serial	PK	identificador
descricao	varchar(30)	NOT NULL, UNIQUE	Comum, Coberta, PCD, Idoso
valor_hora	numeric(6,2)	NOT NULL, CHECK ≥ 0	preço da hora

Dados da carga inicial (DML do script):

id	descricao	valor_hora
1	Comum	6.00
2	Coberta	9.00
3	PCD	0.00
4	Idoso	0.00

vaga

Vaga física do pátio (1 a 8). Guarda só o **tipo_vaga_id** — a tarifa é buscada pela FK.

Coluna	Tipo	Restrições	Significado
id	serial	PK	identificador
numero	integer	NOT NULL, UNIQUE	número pintado no chão
tipo_vaga_id	integer	NOT NULL, FK → tipo_vaga	tipo/tarifa
ativa	boolean	NOT NULL, default true	vaga em uso ou interditada

Dados da carga inicial (DML do script):

id	numero	tipo_vaga_id	(tipo)	ativa
1	1	1	Comum	t
2	2	1	Comum	t
3	3	1	Comum	t
4	4	2	Coberta	t
5	5	2	Coberta	t
6	6	3	PCD	t
7	7	4	Idoso	t
8	8	1	Comum	t

cliente

Condutor/proprietário cadastrado. Um cliente pode ter vários veículos (1:N).

Coluna	Tipo	Restrições	Significado
id	serial	PK	identificador
nome	varchar(100)	NOT NULL	nome completo
cpf	char(11)	NOT NULL, UNIQUE	só números
telefone	varchar(20)	—	contato
email	varchar(100)	UNIQUE	contato

Dados da carga inicial (DML do script):

id	nome	cpf	telefone
1	Ana Beatriz Souza	11111111111	(31) 90000-0001
2	Bruno Carvalho Lima	22222222222	(31) 90000-0002
3	Carla Mendes Ferreira	33333333333	(31) 90000-0003
4	Diego Martins Rocha	44444444444	(31) 90000-0004
5	Elisa Nogueira Pinto	55555555555	(31) 90000-0005
6	Felipe Andrade Costa	66666666666	(31) 90000-0006

funcionario

Atendente que registra a entrada/saída. Os nomes da carga inicial são de integrantes do grupo (dados fictícios).

Coluna	Tipo	Restrições	Significado
id	serial	PK	identificador
nome	varchar(100)	NOT NULL	
cargo	varchar(50)	NOT NULL	Atendente / Supervisor
cpf	char(11)	NOT NULL, UNIQUE	

Dados da carga inicial (DML do script):

id	nome	cargo	cpf
1	João Vitor Alves Rodrigues	Atendente	77777777777
2	Rafael Luiz Ferreira de Souza	Atendente	88888888888
3	Pietro Cardoso de Oliveira	Supervisor	99999999999

veiculo

Veículo de um cliente. **user_id** diz a qual conta do site o veículo pertence (o site cria veículos com placas aleatórias durante a simulação).

Coluna	Tipo	Restrições	Significado
id	serial	PK	identificador
placa	varchar(8)	NOT NULL, UNIQUE	ex.: ABC1D23
cor	varchar(20)	NOT NULL	
modelo	varchar(50)	—	Onix, HB20…
cliente_id	integer	NOT NULL, FK → cliente	dono
user_id	uuid	FK → auth.users, CASCADE	conta do site (nulo na carga inicial)

Dados da carga inicial (DML do script):

id	placa	cor	modelo	cliente_id	user_id
1	ABC1D23	prata	Onix	1	(nulo)
2	DEF4E56	preto	HB20	2	(nulo)
3	GHI7F89	branco	Corolla	3	(nulo)

movimentacao

Coração do sistema: cada linha é uma **entrada e saída** de um veículo (o “pedido” do domínio). Liga veículo, vaga e funcionário.

Coluna	Tipo	Restrições	Significado
id	bigserial	PK	identificador
veiculo_id	integer	NOT NULL, FK → veiculo, CASCADE	qual carro
vaga_id	integer	NOT NULL, FK → vaga, RESTRICT	qual vaga
funcionario_id	integer	NOT NULL, FK → funcionario, RESTRICT	quem registrou
sessao	integer	NOT NULL, default 1	rodada da simulação no site
user_id	uuid	FK → auth.users, CASCADE	conta dona do registro
hora_entrada	timestamptz	NOT NULL, default now()	
hora_saida	timestamptz	CHECK ≥ hora_entrada	nulo = ainda estacionado
tempo_minutos	integer	calculado pela trigger	
valor_cobrado	numeric(8,2)	calculado pela trigger	
status	varchar(15)	NOT NULL, CHECK	'ESTACIONADO' ou 'SAIU'

Dados da carga inicial (DML do script):

id	veiculo	vaga	func.	entrada	saída	min	valor	status
1	1	1	1	17:20	18:20	60	6.00	SAIU
2	2	2	2	18:35	—	—	—	ESTACIONADO
3	3	3	3	18:05	18:50	45	4.50	SAIU

Na movimentação 3 o UPDATE do script registrou a saída e a trigger calculou 45 min e R$ 4,50 (45/60 × R$ 6,00). As linhas da carga inicial têm user_id nulo, por isso não aparecem no painel do site.

SEÇÃO 5

Relacionamentos, restrições e índices

Relacionamentos (todos 1:N)

Lado 1	Lado N (tem a FK)	Coluna FK	ON DELETE	Regra de negócio
tipo_vaga	vaga	tipo_vaga_id	(padrão: bloqueia)	um tipo classifica várias vagas
cliente	veiculo	cliente_id	(padrão: bloqueia)	um cliente tem vários veículos
veiculo	movimentacao	veiculo_id	CASCADE	um veículo gera várias movimentações; apagar o veículo apaga o histórico dele
vaga	movimentacao	vaga_id	RESTRICT	não dá para apagar vaga que já foi usada
funcionario	movimentacao	funcionario_id	RESTRICT	não dá para apagar atendente com registros
auth.users	veiculo	user_id	CASCADE	apagar a conta apaga os veículos dela
auth.users	movimentacao	user_id	CASCADE	apagar a conta apaga as movimentações dela

UNIQUE e CHECK

Tabela	Restrição	O que garante
tipo_vaga	UNIQUE(descricao) · CHECK(valor_hora ≥ 0)	sem tipo repetido; preço nunca negativo
cliente	UNIQUE(cpf) · UNIQUE(email)	um cadastro por pessoa
funcionario	UNIQUE(cpf)	um cadastro por atendente
vaga	UNIQUE(numero)	não existem duas vagas 3
veiculo	UNIQUE(placa)	cada placa uma vez
movimentacao	CHECK(status IN ('ESTACIONADO','SAIU'))	status só com valores válidos
movimentacao	chk_saida_apos_entrada: hora_saida IS NULL OR hora_saida ≥ hora_entrada	não existe saída antes da entrada

Índices (17)

O PostgreSQL cria um índice automaticamente para cada PK (6) e cada UNIQUE (6). Os outros 5 foram criados no script para acelerar os filtros e JOINs mais usados:

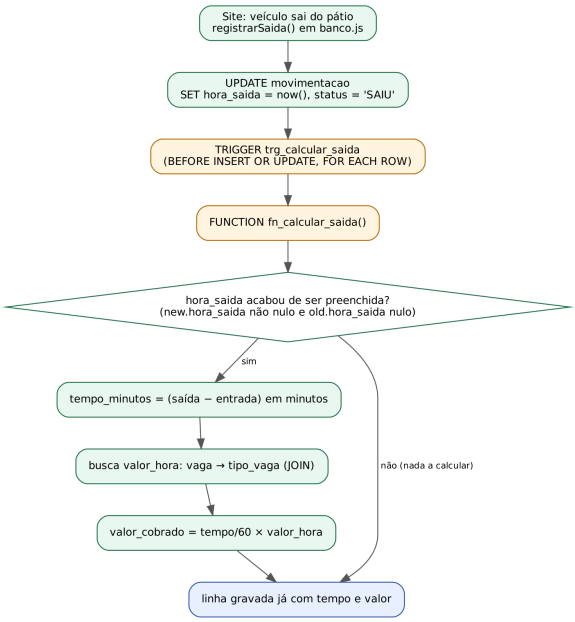
Índice	Tabela(coluna)	Por quê
idx_movimentacao_sessao	movimentacao(sessao)	filtrar a rodada da simulação
idx_movimentacao_veiculo	movimentacao(veiculo_id)	JOIN com veiculo / achar a saída do carro
idx_movimentacao_vaga	movimentacao(vaga_id)	JOIN com vaga / vaga mais usada
idx_veiculo_user	veiculo(user_id)	políticas RLS e filtro por conta
idx_movimentacao_user	movimentacao(user_id)	políticas RLS e painel da conta

Também existem 6 **sequences** (uma por coluna serial/bigserial), que geram os ids 1, 2, 3... automaticamente.

SEÇÃO 6

Function e Trigger: a cobrança calculada pelo banco

É a parte que mais impressiona na apresentação: **o site não calcula o valor**. Ele só informa a hora de saída e o próprio banco calcula o tempo e o valor. Assim a regra fica em um único lugar e nenhum cliente (site, SQL Editor, outro sistema) consegue gravar um valor errado.



Function = um bloco de código guardado no banco (PL/pgSQL).

Trigger = "gatilho" que chama a function automaticamente quando algo acontece na tabela.

Aqui: **antes** de cada INSERT ou UPDATE em `movimentacao`, para cada linha, roda `fn_calcular_saida()`.

Exemplo: entrada 17h20, saída 18h20 em vaga Comum →
tempo = 60 min · valor = 60/60 × 6,00 = **R$ 6,00**.

No site, 1 segundo da simulação = 1 minuto de estadia, então um carro que fica 60 s no pátio paga R$ 6,00.

Vaga PCD/Idoso: valor_hora = 0 → valor_cobrado = R$ 0,00 (tarifa social, RN07).

Por que também no INSERT? Para a carga inicial, que insere uma movimentação já com saída, também sair com tempo e valor calculados.

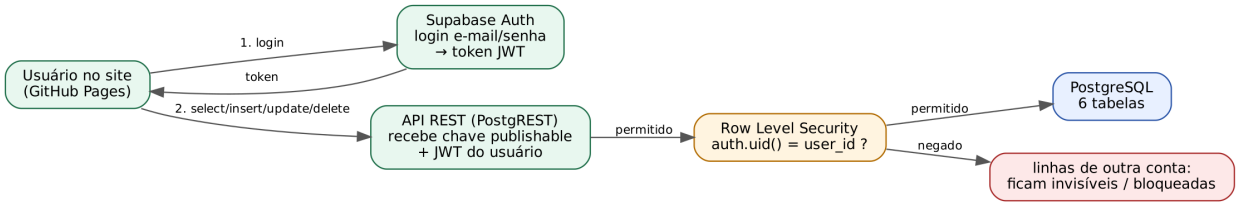
```
create or replace function fn_calcular_saida()
returns trigger as $$
declare
    v_valor_hora numeric(6,2);
begin
    if new.hora_saida is not null and old.hora_saida is null then
        new.tempo_minutos := round(extract(epoch from (new.hora_saida - new.hora_entrada)) / 60.0);
        select tv.valor_hora into v_valor_hora
          from vaga v join tipo_vaga tv on tv.id = v.tipo_vaga_id
         where v.id = new.vaga_id;
        new.valor_cobrado := round((new.tempo_minutos / 60.0) * coalesce(v_valor_hora, 0), 2);
    end if;
    return new;
end;
$$ language plpgsql;

create trigger trg_calcular_saida before insert or update on movimentacao
    for each row execute function fn_calcular_saida();
```

`new` = a linha como vai ficar; `old` = a linha antes da alteração (no INSERT ela não existe, então vale nulo). `extract(epoch …)` devolve a diferença em segundos; dividir por 60 dá minutos.

SEÇÃO 7

Segurança: Auth + Row Level Security



O site usa uma chave pública (*publishable/anon*), que qualquer pessoa pode ver no código. Isso é seguro porque **quem decide o acesso é o banco**: o RLS está ligado nas 6 tabelas e cada operação só passa se uma política permitir. `auth.uid()` é a função do Supabase que devolve o id do usuário logado (ou nulo).

Tabela	Política	Operação	Regra (SQL)	Em português
cliente	leitura_publica_cliente	SELECT	true	qualquer um lê (tabela de referência)
funcionario	leitura_publica_funcionario	SELECT	true	qualquer um lê (tabela de referência)
movimentacao	insercao_conta_movimentacao	INSERT	user_id = auth.uid() (e usuário logado)	só insere com o próprio user_id
movimentacao	leitura_restrita_movimentacao	SELECT	user_id = auth.uid() (e usuário logado)	só lê as próprias movimentações
movimentacao	atualizacao_conta_movimentacao	UPDATE	user_id = auth.uid() (e usuário logado)	só atualiza as próprias linhas
tipo_vaga	leitura_publica_tipo_vaga	SELECT	true	qualquer um lê (tabela de referência)
vaga	leitura_publica_vaga	SELECT	true	qualquer um lê (tabela de referência)
veiculo	exclusao_conta_veiculo	DELETE	user_id = auth.uid() (e usuário logado)	só apaga os próprios veículos
veiculo	insercao_conta_veiculo	INSERT	user_id = auth.uid() (e usuário logado)	só insere com o próprio user_id
veiculo	leitura_publica_veiculo	SELECT	true	qualquer um lê (tabela de referência)

- Sem política = proibido. Por isso ninguém consegue, por exemplo, apagar um cliente ou mudar a tarifa pela API do site.
- Visitante (sem login): vê a simulação, mas nada é gravado — o código do site nem tenta, e o RLS bloquearia de qualquer forma.
- Botão **Reiniciar**: `delete from veiculo where user_id = auth.uid();` o ON DELETE CASCADE apaga as movimentações junto.
- Usuário de teste do Supabase Auth: **admin@parksim.com** / admin123 (aparece em Authentication → Users).

SEÇÃO 8

Como o site conversa com o banco

Toda a comunicação está em `js/banco.js`. O JavaScript usa a biblioteca supabase-js, que transforma cada chamada em SQL no PostgreSQL:

Ação no site	Função (banco.js)	SQL equivalente executado no banco
Abrir o site	carregarReferencias()	SELECT id, numero FROM vaga; SELECT id FROM funcionario; SELECT id FROM cliente
Login	login()	Supabase Auth valida e-mail/senha e devolve o token (auth.uid())
Carro entra	obterOuCriarVeiculo() + registrarEntrada()	INSERT INTO veiculo (…, user_id) se a placa for nova; INSERT INTO movimentacao (veiculo_id, vaga_id, funcionario_id, sessao, user_id, 'ESTACIONADO')
Carro sai	registrarSaida()	SELECT da movimentação aberta; UPDATE movimentacao SET hora_saida = hora_entrada + permanência, status = 'SAIU' → dispara a trigger
Painel / PDF	buscarMovimentacoes()	SELECT com JOIN movimentacao + veiculo + cliente + vaga + funcionario WHERE user_id = auth.uid()
Reiniciar	limparMovimentacoes()	DELETE FROM veiculo WHERE user_id = auth.uid() (CASCADE limpa movimentacao)

Tempo da simulação: cada segundo no pátio vale 1 minuto de estadia (o relógio do site mostra os minutos simulados). O site só informa a hora de saída; tempo_minutos e valor_cobrado continuam sendo calculados pela trigger. Ex.: 69 s numa vaga Coberta → 69 min → R$ 10,35.

O arquivo .sql da entrega (o que o professor pediu)

Parte do script	Comandos	O que faz
1. DDL	DROP/CREATE TABLE, CREATE INDEX, CREATE FUNCTION, CREATE TRIGGER, ALTER TABLE … ENABLE RLS, CREATE POLICY	cria toda a estrutura
2. DDL (migração)	ALTER TABLE … ADD COLUMN IF NOT EXISTS user_id, CREATE INDEX IF NOT EXISTS, DROP POLICY IF EXISTS	evolução real feita no banco publicado (idempotente: pode rodar de novo sem erro)
3. DML	INSERT (4 tipos, 8 vagas, 6 clientes, 3 funcionários, 3 veículos, 3 movimentações), UPDATE, DELETE	carga inicial fictícia para homologação
4. DQL	6 SELECTs com JOIN, COUNT, SUM, AVG, GROUP BY	relatórios estratégicos

Testado: o script completo roda de ponta a ponta sem erro em PostgreSQL (com o schema auth do Supabase simulado). No Supabase basta colar no SQL Editor e clicar em **Run**. Atenção: a parte 1 começa com DROP TABLE — rodar no projeto publicado apaga os dados atuais.

SEÇÃO 8 (CONT.)

As consultas DQL e o que elas devolvem

Resultados reais da execução do script (carga inicial). Bom para mostrar na apresentação que as consultas funcionam:

4.2 Faturamento por cliente no último mês — o exemplo do enunciado (“notas fiscais dos clientes no último mês”)

cliente	movimentações	faturamento (R$)
Ana Beatriz Souza	1	6.00
Carla Mendes Ferreira	1	4.50

4.3 Vaga mais utilizada (LEFT JOIN mostra também as vagas nunca usadas)

vaga	tipo	vezes usada
1	Comum	1
2	Comum	1
3	Comum	1
4	Coberta	0
5	Coberta	0
6	PCD	0
7	Idoso	0
8	Comum	0

4.4 Tempo médio de permanência por tipo de vaga

tipo	média (min)
Comum	52.5

4.5 Funcionário que mais registrou movimentações

nome	cargo	atendimentos
João Vitor Alves Rodrigues	Atendente	1
Rafael Luiz Ferreira de Souza	Atendente	1
Pietro Cardoso de Oliveira	Supervisor	1

4.6 Veículos estacionados agora

placa	cor	cliente	vaga
DEF4E56	preto	Bruno Carvalho Lima	2

A 4.1 (relatório completo com JOIN das 6 tabelas) é a mesma consulta que alimenta o painel do site e o PDF exportado.

SEÇÃO 8 (CONT.)

Diagramas oficiais do projeto e normalização

Figura — DER conceitual (notação de Chen), igual ao documento e aos slides

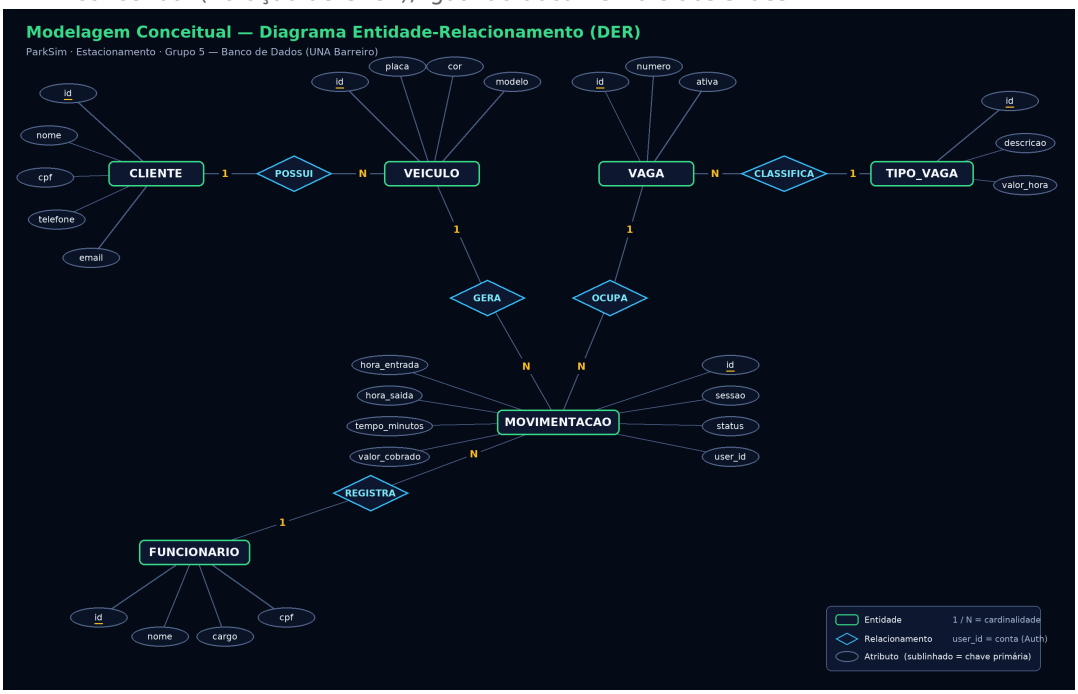


Figura — Modelo lógico relacional com PK/FK

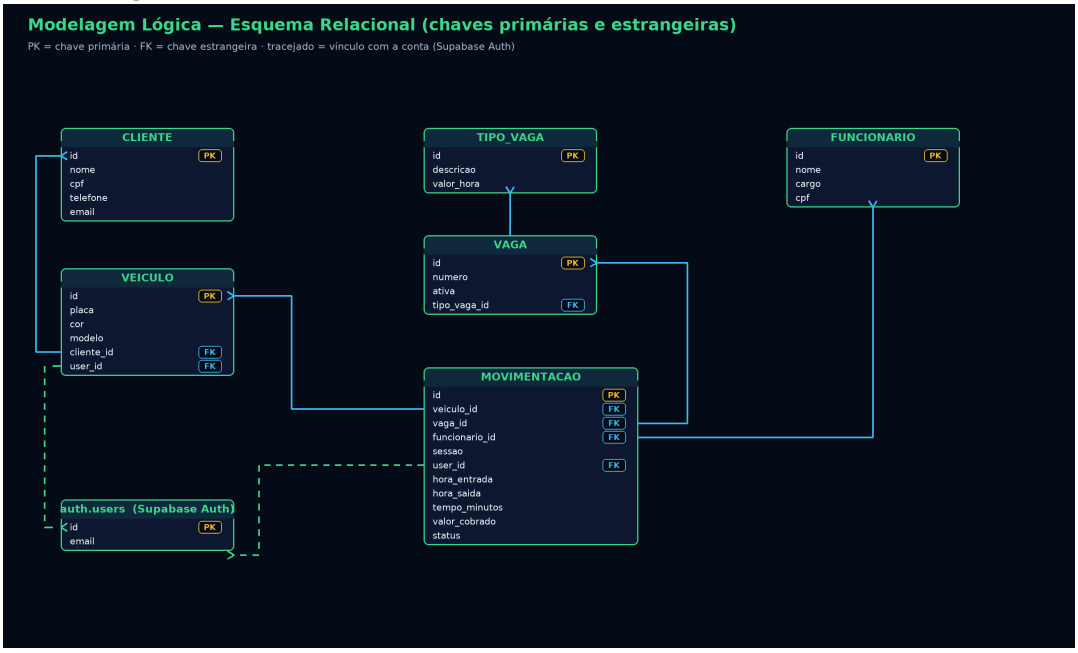
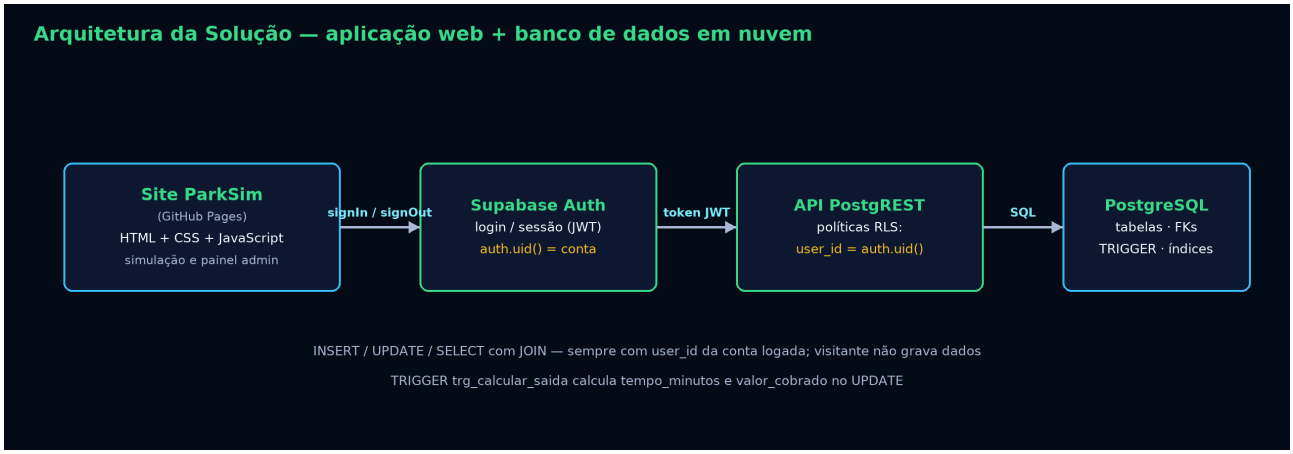


Figura — Arquitetura da solução



Normalização em linguagem simples

Forma normal	Regra	Como o ParkSim cumpre
1FN	valores atômicos, sem listas dentro de uma célula, toda tabela com PK	cada entrada é uma linha em movimentacao (nunca “entradas: 3, 5, 9” numa célula)
2FN	nada depende de só uma parte de uma chave composta	todas as PKs são simples (id), então não há dependência parcial
3FN	nenhuma coluna não-chave depende de outra coluna não-chave	a tarifa está em tipo_vaga, não em vaga nem em movimentacao; nome do cliente/placa/atendente entram por FK

Ponto que o professor pode questionar: `movimentacao.user_id` repete o `user_id` do veículo. É uma **desnormalização consciente**: permite que a política RLS filtre cada linha direto (user_id = auth.uid()) sem subconsulta e deixa o painel mais rápido. `tempo_minutos` e `valor_cobrado` também são derivados, mas guardados como histórico da cobrança (se a tarifa mudar amanhã, o valor cobrado ontem não muda) e só a trigger os escreve.

SEÇÃO 9

Treino: perguntas prováveis e respostas

P: Quantas tabelas o banco tem e quais são?

R: Seis: tipo_vaga, vaga, cliente, funcionario, veiculo e movimentacao. Além delas o Supabase tem a auth.users, que é do serviço de login e não foi criada por nós.

P: Qual é a tabela principal e por quê?

R: movimentacao: é o fato do negócio (cada entrada/saída). Ela liga veículo, vaga e funcionário por três FKs e guarda horários, tempo, valor e status.

P: Mostre um relacionamento 1:N.

R: cliente → veiculo: um cliente tem vários veículos; a FK cliente_id fica no lado N (veiculo). Mesmo raciocínio para veiculo → movimentacao.

P: Por que existe a tabela tipo_vaga?

R: Para a tarifa não se repetir em cada vaga (3FN). Se o preço da vaga Comum mudar, alteramos uma linha só, sem anomalia de atualização.

P: O que é a trigger e o que ela faz?

R: Um gatilho na movimentacao que, antes de gravar, chama fn_calcular_saida(). Quando a hora de saída é preenchida, calcula os minutos e o valor (minutos/60 × valor_hora do tipo da vaga).

P: Por que calcular no banco e não no site?

R: Regra em um lugar só, valor confiável e impossível de forjar pelo navegador; qualquer sistema que grave no banco recebe o mesmo cálculo.

P: O que acontece se apagar um veículo? E uma vaga?

R: Veículo: ON DELETE CASCADE apaga as movimentações dele. Vaga: ON DELETE RESTRICT bloqueia se ela já tiver movimentações, para não perder histórico.

P: Como garantem que uma saída não é anterior à entrada?

R: Pela CHECK chk_saida_apos_entrada: hora_saida IS NULL OR hora_saida >= hora_entrada. O banco recusa o dado.

P: A chave do Supabase está no código do site. Não é inseguro?

R: É a chave pública (anon/publishable), feita para ficar no navegador. Quem protege os dados é o RLS: sem política que permita, o banco nega.

P: O que é RLS?

R: Row Level Security: regras por linha. Ex.: em movimentacao só é possível ler/inserir/atualizar linhas com user_id = auth.uid(), a conta logada.

P: Onde estão DDL, DML e DQL?

R: No arquivo database/parksim_schema.sql: seção 1 e 2 (DDL), seção 3 (DML: INSERT/UPDATE/DELETE) e seção 4 (DQL: SELECTs com JOIN).

P: Qual consulta atende ao exemplo do professor (notas dos clientes no último mês)?

R: A 4.2: JOIN cliente–veiculo–movimentacao, filtra hora_saida no último mês e soma valor_cobrado por cliente (GROUP BY).

P: Por que serial em vez de usar CPF ou placa como chave?

R: Chave substituta: curta, nunca muda e não expõe dado pessoal. CPF e placa continuam protegidos por UNIQUE.

P: Por que o painel do site não mostra os dados da carga inicial?

R: Porque eles têm user_id nulo e a política de leitura de movimentacao só mostra as linhas da conta logada.

SEÇÃO 10

Roteiro sugerido para a apresentação (6 partes)

Segue a ordem pedida no enunciado dos professores. Distribuam as partes entre os integrantes; os slides correspondentes estão em `entrega/ParkSim_A3_Apresentacao.pptx`.

Parte	Conteúdo	Slides	O que mostrar ao vivo
1	Contexto, problema e solução; atividades e informações armazenadas	1–5	—
2	Consultas/relatórios necessários e regras de negócio (RN01–RN09)	6–7	—
3	Modelagem conceitual: DER, entidades, PKs e cardinalidades	8–9	Schema Visualizer no Supabase
4	Modelagem lógica, normalização até 3FN e integridade referencial	10–13	Table Editor: abrir vaga e tipo_vaga
5	Modelagem física: DDL, trigger, DML e DQL	14–17	SQL Editor: rodar a consulta 4.2; Database → Triggers
6	Site, arquitetura, segurança (Auth + RLS) e conclusão	18–22	Site: login, carro entra/sai, painel; depois Table Editor → movimentacao com valor calculado

Checklist da demonstração ao vivo

- Na véspera, abrir o projeto no Supabase: em plano gratuito ele **hiberna** após dias sem uso e leva alguns minutos para voltar.
- Entrar no site com admin@parksim.com, deixar entrar e sair alguns carros (use 3x) e abrir o painel: as saídas mostram minutos e valores calculados pelo banco.
- No Supabase → Table Editor → movimentacao: mostrar a linha nova com tempo_minutos e valor_cobrado preenchidos pela trigger.
- Ter a consulta 4.2 já colada no SQL Editor para rodar com um clique.
- Não rodar o script inteiro no projeto publicado durante a apresentação (ele começa com DROP TABLE).

Glossário rápido

Termo	Significado
DDL	comandos que criam/alteram estrutura: CREATE, ALTER, DROP
DML	comandos que mexem nos dados: INSERT, UPDATE, DELETE
DQL	consultas: SELECT
JOIN	junta linhas de tabelas diferentes pela FK
serial	inteiro que se auto-incrementa (usa uma sequence)
timestamptz	data e hora com fuso horário
PL/pgSQL	linguagem de programação dentro do PostgreSQL (usada na function)
PostgREST	API do Supabase que transforma chamadas HTTP do site em SQL
auth.uid()	função do Supabase que devolve o id do usuário logado